

Capa de transporte

UDP y TCP

Programación y administración de redes

Grado en *Ingeniería Informática*

Departamento de Informática. Universidad de Jaén

Objetivos

General

Situar la capa de transporte en la arquitectura de redes de Internet e identificar sus funciones, así como conocer sus protocolos fundamentales

Específicos

- Determinar cómo se lleva a cabo la **comunicación al nivel de capa** de transporte
- Conocer las **características de UDP** y el formato de su segmento
- Analizar el **funcionamiento de TCP**, conocer sus características y formato de su segmento
- Identificar las **diferencias fundamentales** entre UDP y TCP
- Conocer el mecanismo de **multiplexación** en UDP y en TCP

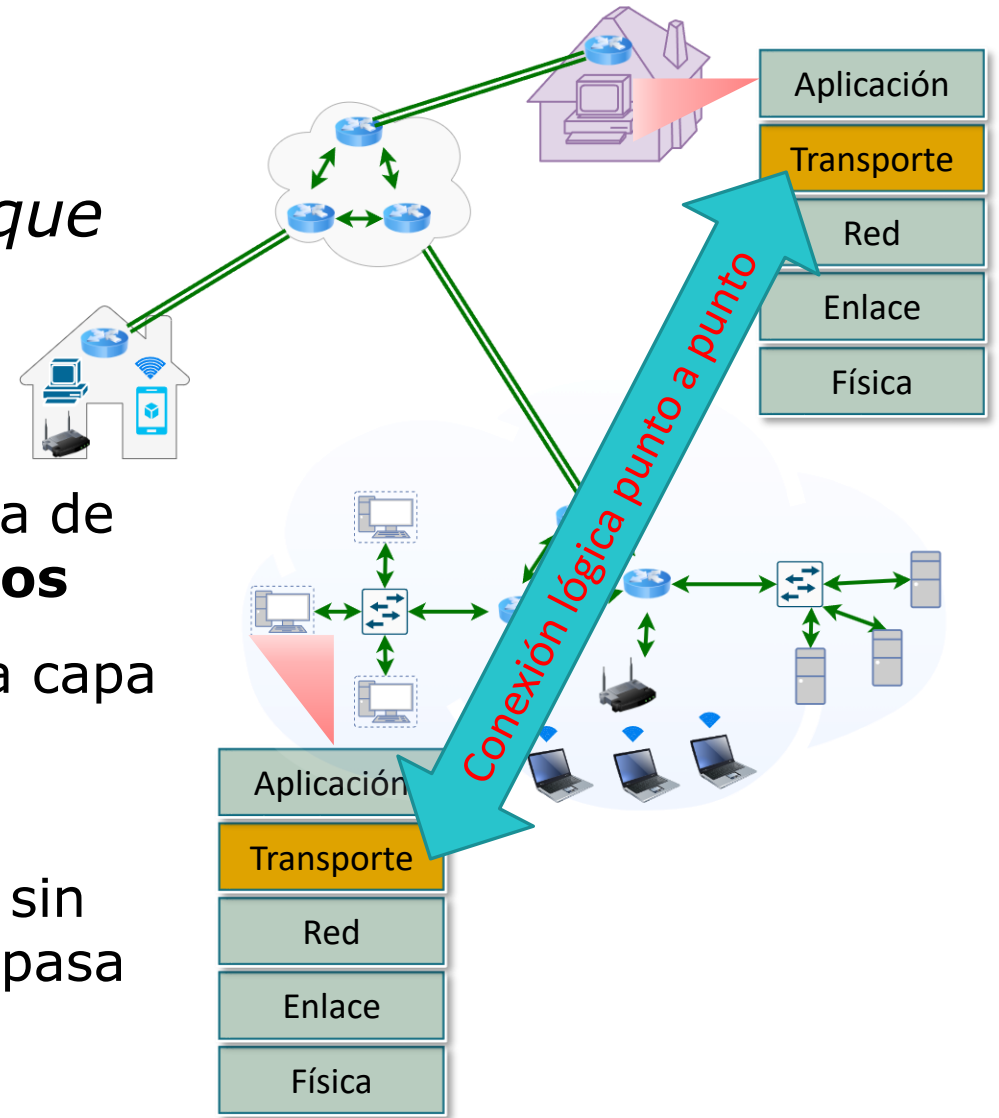
Introducción

Finalidad de la capa de transporte

Proporcionar una comunicación lógica de extremo a extremo a procesos de aplicación que se ejecutan en diferentes host

Cómo funciona

- La capa de transporte se sitúa debajo de la capa de aplicación, por lo que ofrece a esta **sus servicios**
- Los puntos de acceso (SAP) a los servicios de la capa de transporte se denominan **sockets**
- Los procesos de la capa de aplicación pueden comunicarse entre sí de **forma transparente**, sin conocer detalles sobre las redes por las que se pasa
- La capa de transporte es la responsable del mecanismo de **multiplexación**



Protocolos de la capa de transporte

Finalidad

Establecer el mecanismo que permita identificar a los procesos que envían y que han de recibir los mensajes enviados por la capa de aplicación

Detalles

— Dos protocolos

- UDP: ofrece un servicio **no orientado a conexión**
- TCP: ofrece un servicio **orientado a conexión**

UDP
User Datagram Protocol

— *Payload*

- Toman como paquete de datos los **mensajes** facilitados por los protocolos de la capa de aplicación: HTTP, FTP, SMTP, etc.

TCP
Transmission Control Protocol

— Direccionamiento

- La **cabecera** de estos protocolos es la encargada de identificar de manera única los procesos origen y destino

— Multiplexación

- Cada protocolo define el mecanismo encargado de **multiplexar la conexión** a red hacia/desde varios procesos

El protocolo UDP



El protocolo UDP - Introducción

Definición

*UDP (User Datagram Protocol) es un protocolo de transporte eficiente y **no orientado a conexión** que ofrece un servicio "best-effort"*

Características

- Se define en el **RFC 768** (<https://tools.ietf.org/html/rfc768>) que no ha sufrido cambio alguno desde que se publicó en 1980
- Añade **muy pocas funcionalidades**, la especificación solo tiene tres páginas, limitándose a entregar los datos de la capa de aplicación a la capa de red
- Ofrece un servicio de tipo **no orientado a conexión**
- El servicio es **no fiable**: se pueden perder paquetes
- No se garantiza que los paquetes lleguen en **orden**
- No se realiza ningún control de tráfico ni **congestión**

El protocolo UDP - Limitaciones y usos

Definición

La simplicidad de UDP implica limitaciones, pero a cambio ofrece características deseables para ciertos usos

Detalles

- Sin establecimiento de conexión (lo que **elimina el retardo** asociado)
- Simple: **sin estado** de conexión en el emisor y el receptor (se eliminan *buffers*, variables de estado, ...)
- UDP añade un **encabezamiento muy pequeño**
- Al no haber control de congestión también se **elimina el gasto** asociado

Usos

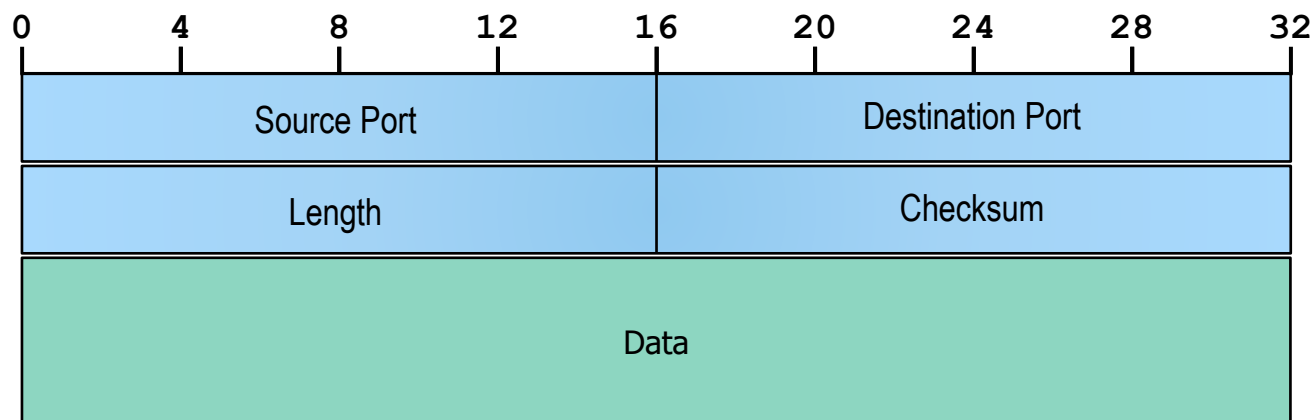
- Frecuentemente utilizado para la transmisión de datos multimedia (vídeo o audioconferencia) donde hay:
 - Tolerancia a pérdidas y mucha sensibilidad a retardos
 - Posibilidad de envíos *unicast*, *multicast* y *broadcast*
- También se usa en protocolos como DNS, SNMP, RIP, algunos de los cuales conoceremos más adelante

Si se quiere tener una transferencia fiable con UDP, debe ser implementada en la capa de aplicación, por ejemplo es el caso de HTTP/3 (<https://www.rfc-editor.org/rfc/rfc9114.html>)

El protocolo UDP - Formato del segmento

Segmento UDP

El segmento UDP toma como payload el mensaje de la capa de aplicación, agrega su cabecera y después se encapsula en un datagrama IP con el campo de protocolo = 17



```
#<netinet/udp.h>
struct
{
    uint16_t source;
    uint16_t dest;
    uint16_t len;
    uint16_t check;
};
```

- **Source Port** (16 bits): identifica al proceso que actúa como emisor
- **Destination Port** (16 bits): identifica al proceso que actúa como receptor
- **Length** (16 bits): longitud, en bytes, del segmento UDP completo, incluida la cabecera
- **Checksum** (16 bits): suma de comprobación de errores. Se calcula sobre todo el contenido, cabecera + datos
- **Data**: *payload* del segmento con los datos entregados por la capa de aplicación

Concepto de puerto

Definición

*El punto de acceso a servicio de la **capa de transporte** está vinculado al concepto de "puerto", con el que se enlaza cada conexión con el proceso correspondiente de la **capa de aplicación***

Detalles

- Los puertos se identifican con un valor entero sin signo de 16 bits: 0 — 65535
- El **puerto de origen** suele ser seleccionado de forma aleatoria
- El **puerto de destino**, por el contrario, ha de ser conocido para que los datos lleguen al proceso adecuado

Uso de puertos

- 0–1023: puertos “bien conocidos”, reservados para servicios concretos
- 1024–49151: puertos registrados, no reservados inicialmente pero que pueden ser solicitados para su uso en aplicaciones concretas
- 49152–65535: puertos de uso privado y dinámico, cualquier programa puede emplearlos libremente

El protocolo TCP Cabecera



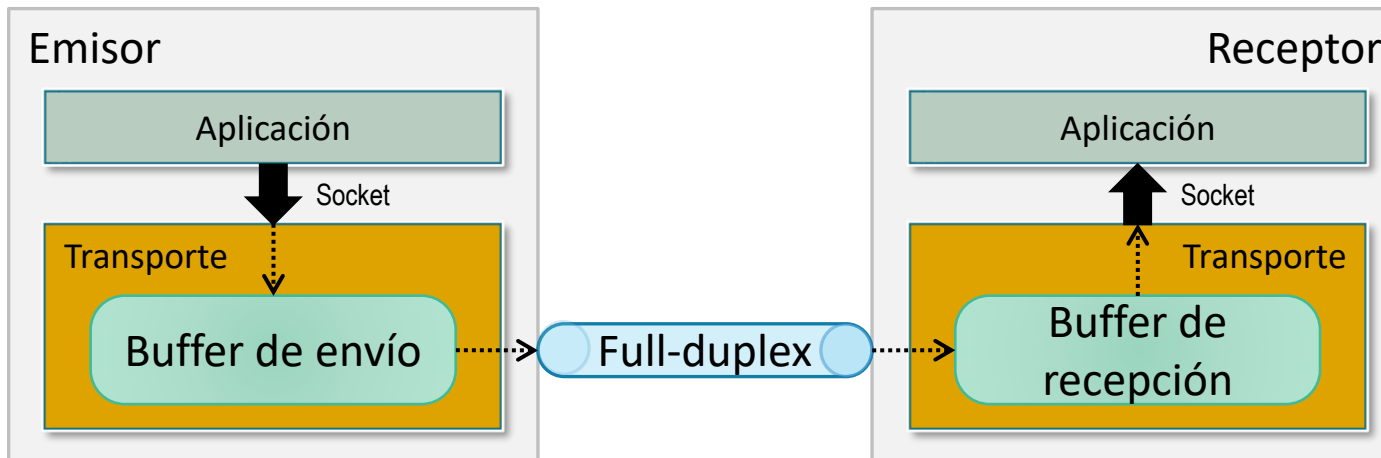
El protocolo TCP - Introducción

Definición

*TCP (Transmission Control Protocol) es un protocolo de transporte **orientado a conexión** y que mantiene estado en los host implicados*

Características

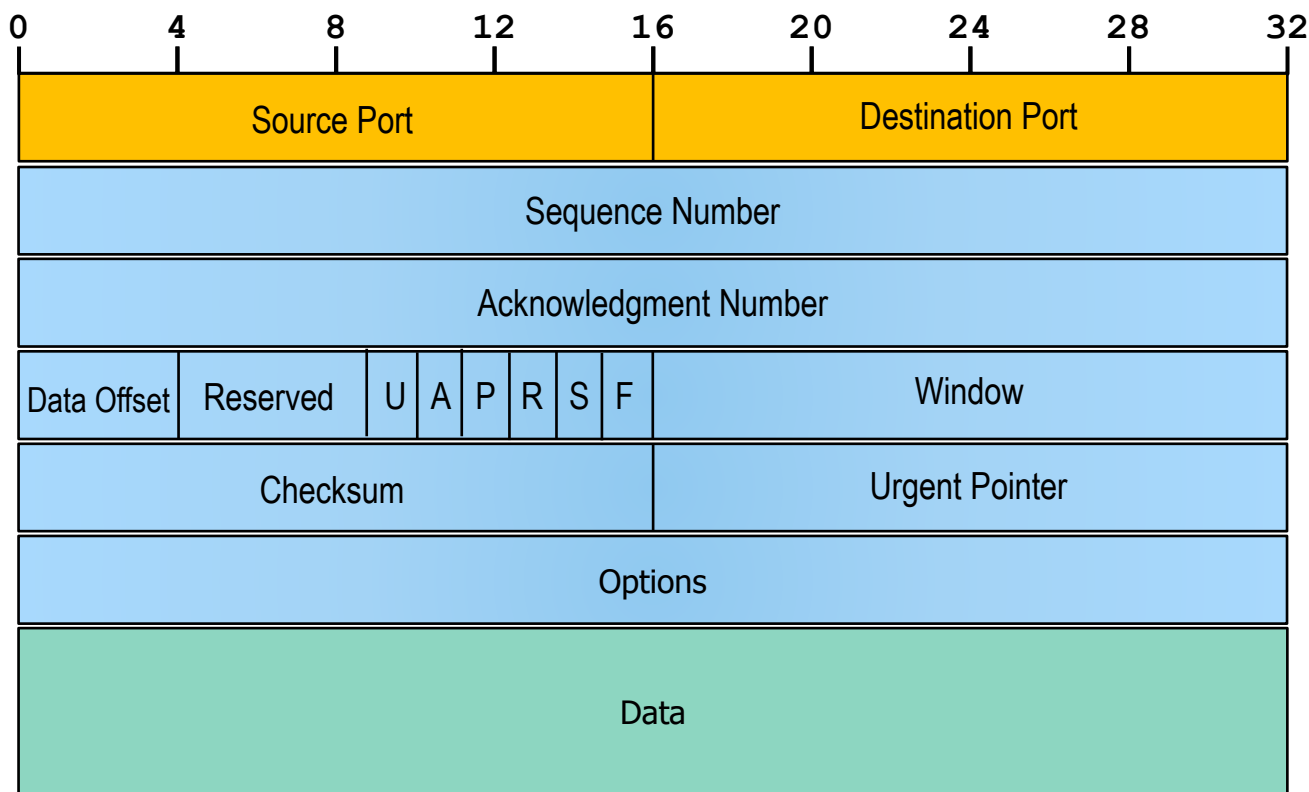
- Definido originalmente en el **RFC 793** (<https://tools.ietf.org/html/rfc793>) y actualizado en los RFC 1122, 3168, 6093 y 6528
- Ofrece un servicio de transporte **fiable**: tiene control de errores, control de flujo y los segmentos se reciben en el orden en que se enviaron
- La comunicación es **punto a punto** (*unicast*, no se permiten conexiones *multicast*) y *full-dúplex*: un canal por el que ambos extremos pueden enviar y recibir información
- Mantiene unos *buffer* en el emisor y otro en el receptor, aunque no especifica cómo se vacían y se llenan



El protocolo TCP - Formato del segmento

Segmento TCP

El segmento TCP toma como payload el mensaje de la capa de aplicación, agrega su cabecera y después se encapsula en un datagrama IP con el campo de protocolo = 6

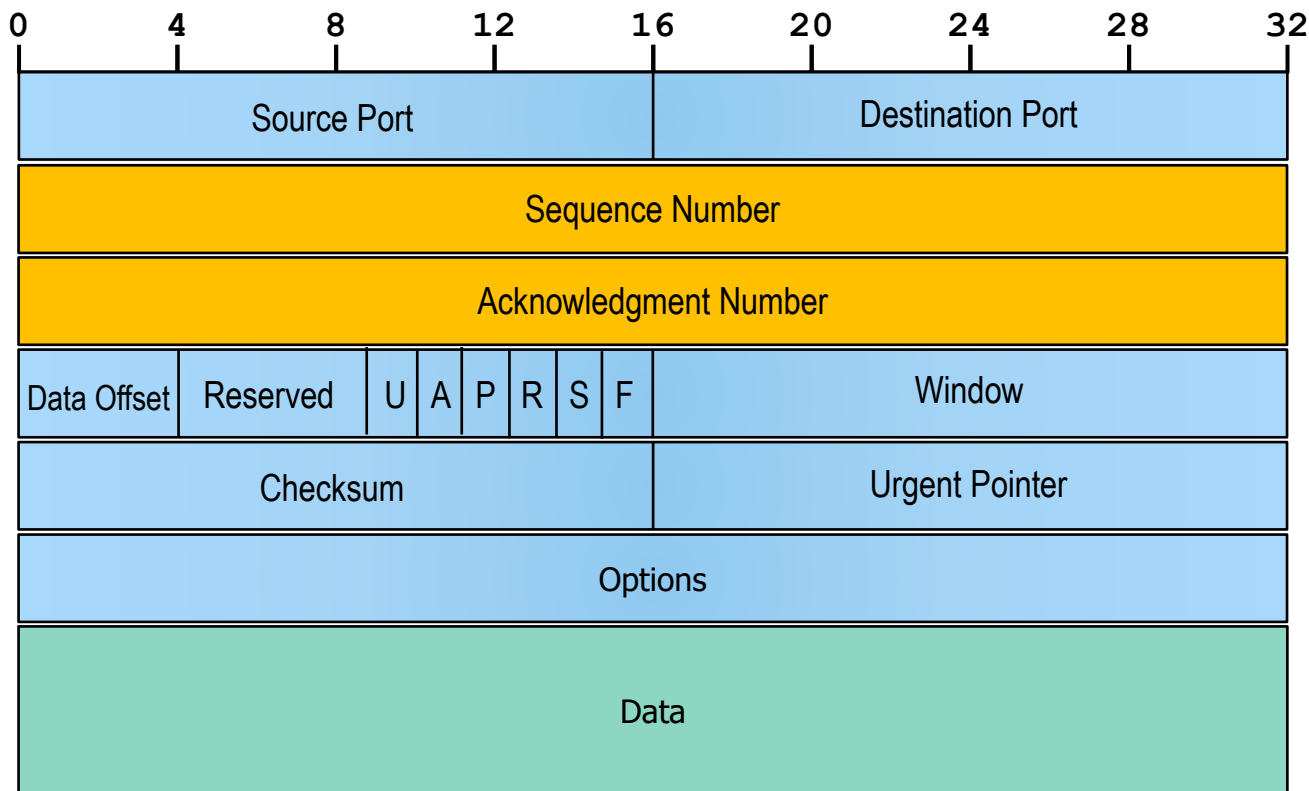


- **Source Port** (16 bits): identifica al proceso que actúa como emisor
- **Destination Port** (16 bits): identifica al proceso que actúa como receptor

El protocolo TCP - Formato del segmento

Segmento TCP

El segmento TCP toma como payload el mensaje de la capa de aplicación, agrega su cabecera y después se encapsula en un datagrama IP con el campo de protocolo = 6



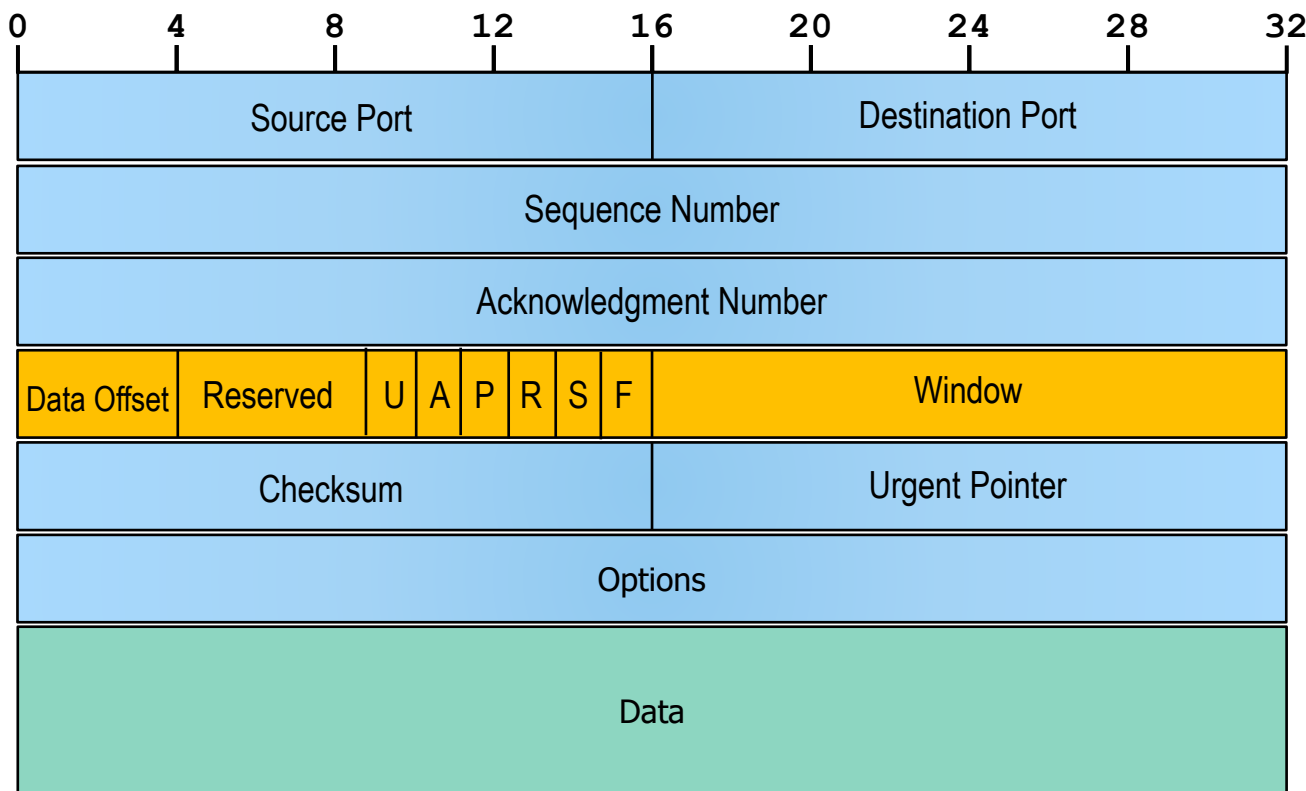
- **Sequence Number** (32 bits): número de secuencia que corresponde a este datagrama dentro del flujo de comunicación. Se parte de un valor inicial, acordado durante la conexión, que se incrementa a medida que los datos se envían
- **Acknowledgment** (32 bits): si el bit **A** está activo, este campo contendrá el siguiente byte que se espera recibir, reconociendo como recibidos los anteriores. Este campo está relacionado con el número de secuencia de la otra parte y viceversa

Nos ocuparemos del funcionamiento de estos dos campos con más detalle en el próximo tema

El protocolo TCP - Formato del segmento

Segmento TCP

El segmento TCP toma como payload el mensaje de la capa de aplicación, agrega su cabecera y después se encapsula en un datagrama IP con el campo de protocolo = 6

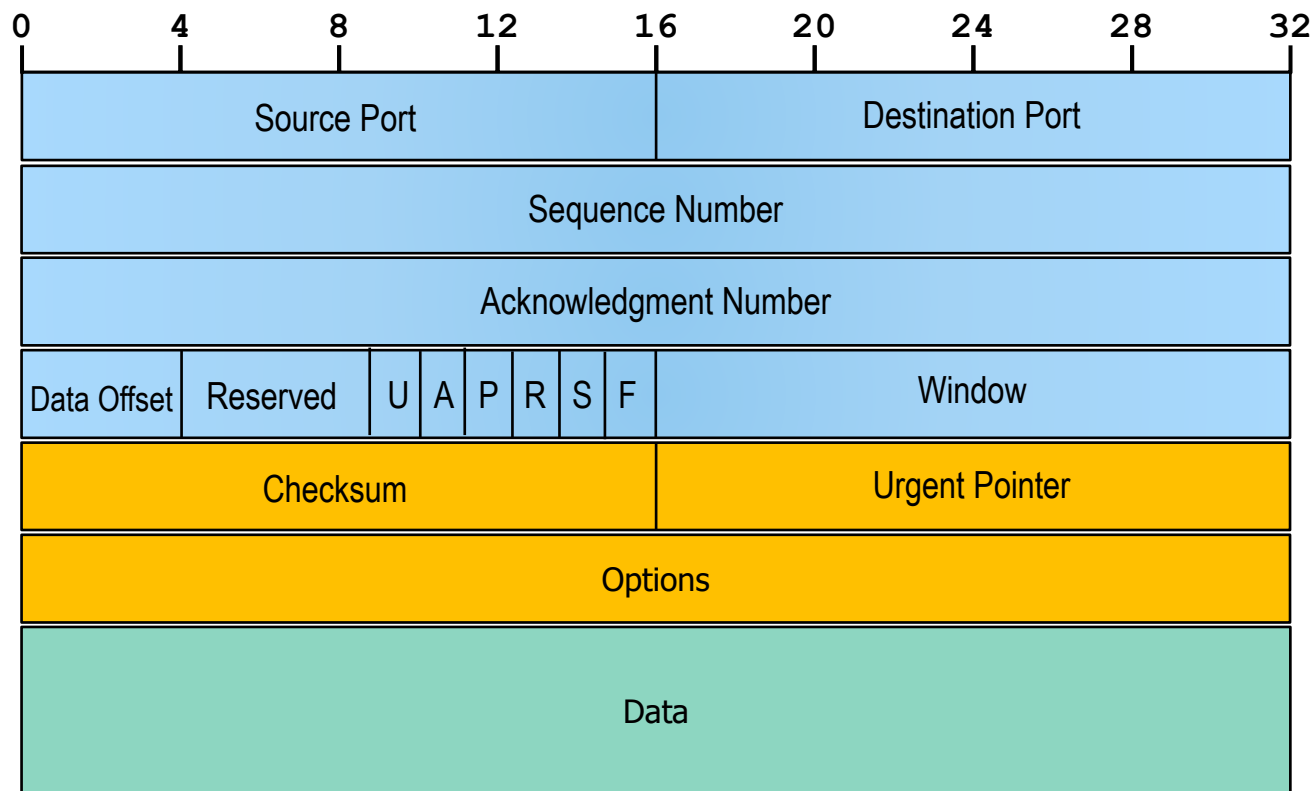


- **Data Offset - Longitud de cabecera:** indicado en palabras (32 bits), ya que la longitud de la cabecera puede variar según las opciones que se incluyan
- **Indicadores:** diversas funciones de control
 - **(U)RG:** datos urgentes de comunicación
 - **(A)CK:** reconocimiento de datos
 - **(P)SH:** enviar de manera inmediata sin esperas
 - **(R)ST:** reiniciar conexión por error
 - **(S)YN:** sincronización de conexión
 - **(F)IN:** finalización de la conexión
- **Window - Ventana recepción:** para control de flujo, el receptor le indica al cliente cuántos bytes tiene libres en el *buffer* de recepción

El protocolo TCP - Formato del segmento

Segmento TCP

El segmento TCP toma como payload el mensaje de la capa de aplicación, agrega su cabecera y después se encapsula en un datagrama IP con el campo de protocolo = 6

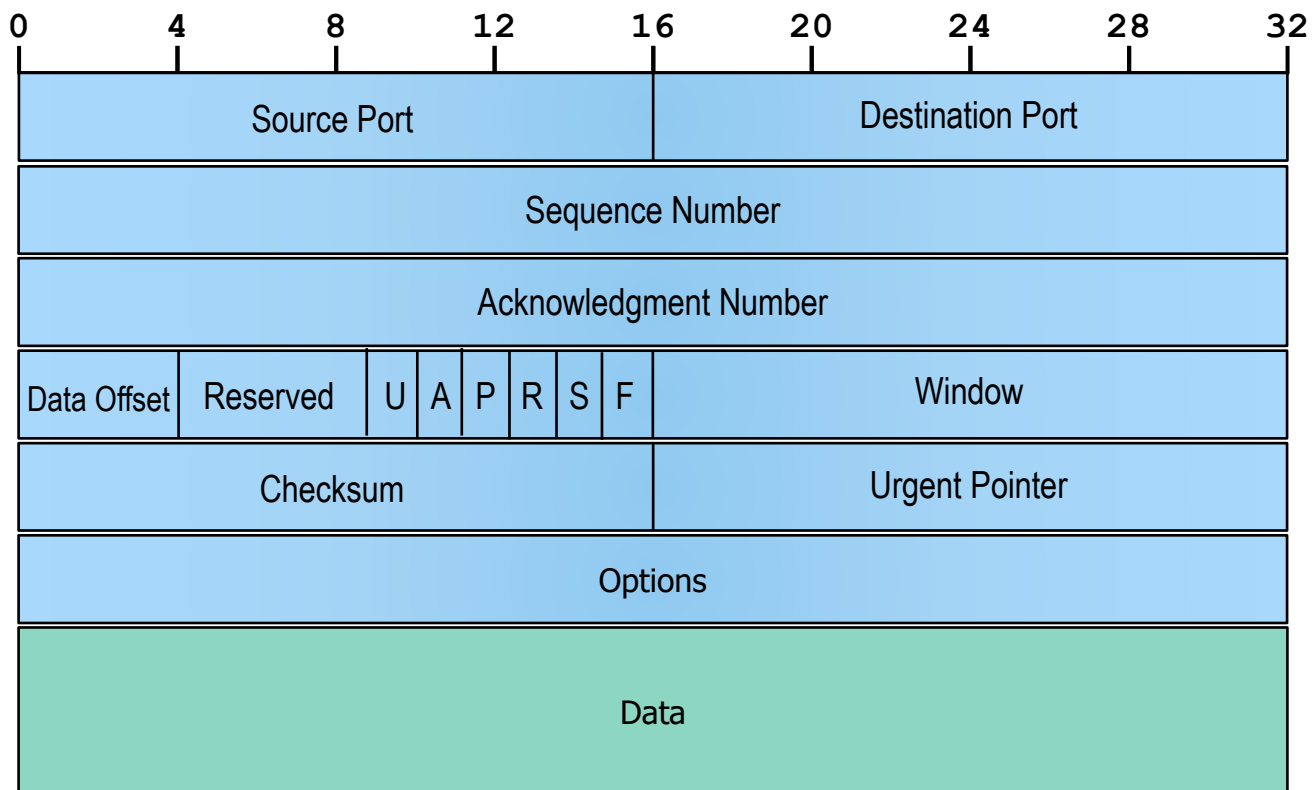


- **Checksum** (16 bits): suma de comprobación de errores. Funciona como en el protocolo UDP
- **Urgent Pointer** (16 bits): si el bit **U** está activo, este campo contendrá el número de secuencia que corresponde al último byte de datos urgentes
- **Options**: como en el caso del protocolo IPv4, TCP también contempla la existencia de una lista de opciones de longitud variable. En TCP se utilizan, entre otras tareas, para negociar el MSS, tamaño de ventana, etc.

El protocolo TCP - Formato del segmento

Segmento TCP – Estructura de datos

La estructura de datos con la que se representa un segmento TCP, definida en el archivo de cabecera `netinet/tcp.h`, es mucho más compleja que la de UDP



```
#<netinet/tcp.h>
struct {
    uint16_t source;
    uint16_t dest;
    uint32_t seq;
    uint32_t ack_seq;
    uint16_t res1:4;
    uint16_t doff:4;
    uint16_t fin:1;
    uint16_t syn:1;
    uint16_t rst:1;
    uint16_t psh:1;
    uint16_t ack:1;
    uint16_t urg:1;
    uint16_t res2:2;
    uint16_t window;
    uint16_t check;
    uint16_t urg_ptr;
};
```


El protocolo TCP

Inicio y fin de conexión



Establecimiento de la conexión

Descripción

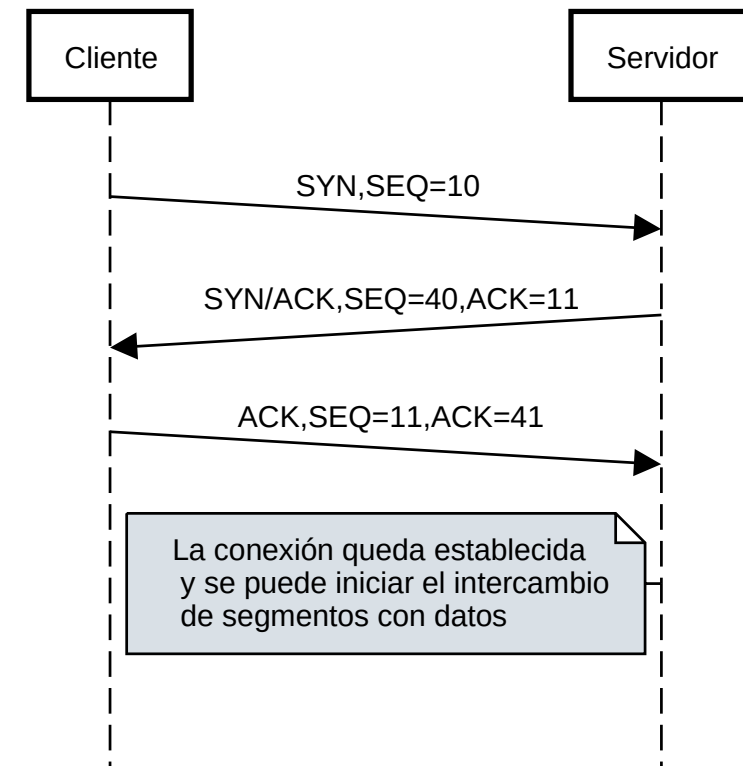
Como servicio orientado a conexión, el protocolo TCP ha de crear un canal de comunicación sincronizando a cliente y servidor a fin de facilitar el intercambio de datos de una manera fiable

Secuencia de establecimiento de conexión:

1. El cliente envía un segmento TCP al servidor activando el bit **S(YN)** e indicando en el campo **SEQ** su número de secuencia (N) inicial
2. El servidor responde activando los bits **S(YN)** y **A(CK)**, incluyendo en el campo **SEQ** su propio número de secuencia (M) y en el campo **ACK** el número de secuencia enviado por el cliente incrementado en una unidad ($N+1$)
3. El cliente asiente para confirmar la conexión, enviando un segmento con el bit **A(CK)** activado e incluyendo en **SEQ** el siguiente valor de su secuencia ($N+1$) y en el campo **ACK** el siguiente valor de la secuencia del servidor ($M+1$)

Una vez establecida la conexión, cliente y servidor intercambian segmentos **empleando los números de secuencia** acordados durante este proceso conocido como *three-way handshake*

Establecimiento de conexión en TCP



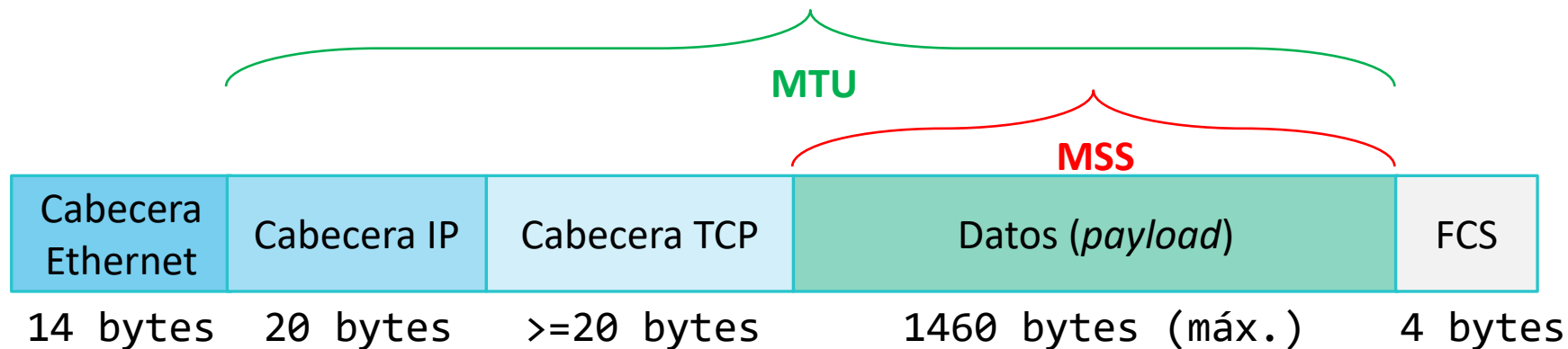
Negociación del MSS

Definición

El tamaño máximo del payload en un segmento TCP depende del MSS (Maximum Segment Size), un parámetro que se negocia durante el establecimiento de conexión, usando el campo de opciones de la cabecera

Conceptos:

- El MSS depende del tamaño de la **MTU** (*Maximum Transfer Unit*) que ofrezca la capa de enlace
- Para la capa de enlace de Ethernet la MTU es de 1500 bytes (otras tecnologías tienen MTU distintas)
- **MSS = MTU - Tamaños de cabeceras** (las cabeceras de TCP tienen 20 bytes o más, dependiendo de las opciones)



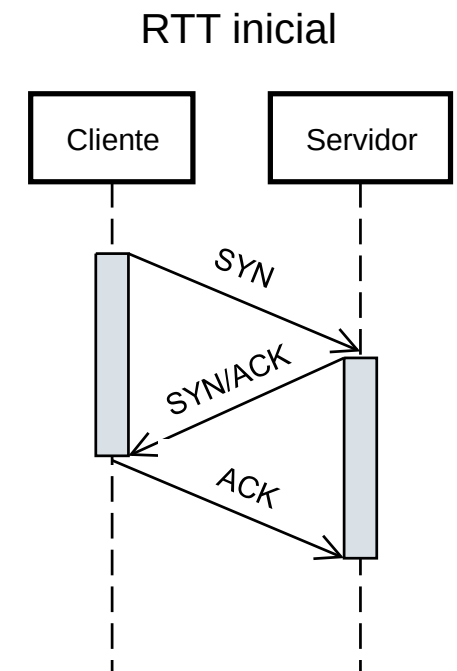
Estimación del RTT

Definición

Se denomina RTT (Round Trip Time), o "tiempo de ida y vuelta", al tiempo que se precisa para que un segmento viaje desde el nodo emisor al receptor y la respuesta llegue de nuevo al emisor

RTT inicial

- El RTT inicial se estima durante el proceso de conexión TCP, según el esquema del diagrama a la derecha
 - El cliente envía el segmento **SYN** y mide el tiempo hasta recibir respuesta
 - El servidor envía el segmento **SYN/ACK** y mide también su tiempo
- Este RTT inicial sirve como base para ajustar distintos parámetros del protocolo TCP, entre ellos:
 - Ajustar la tasa de envío de datos para **evitar la congestión** de la red
 - Fijar los temporizadores de reenvío de paquetes para **evitar pérdidas** de datos



Además de los números de secuencia iniciales, el MSS y el RTT, el proceso de conexión TCP sirve también para acordar otros parámetros, como los tamaños de los búferes de recepción, algoritmos de control de congestión, etc.

Cierre de la conexión

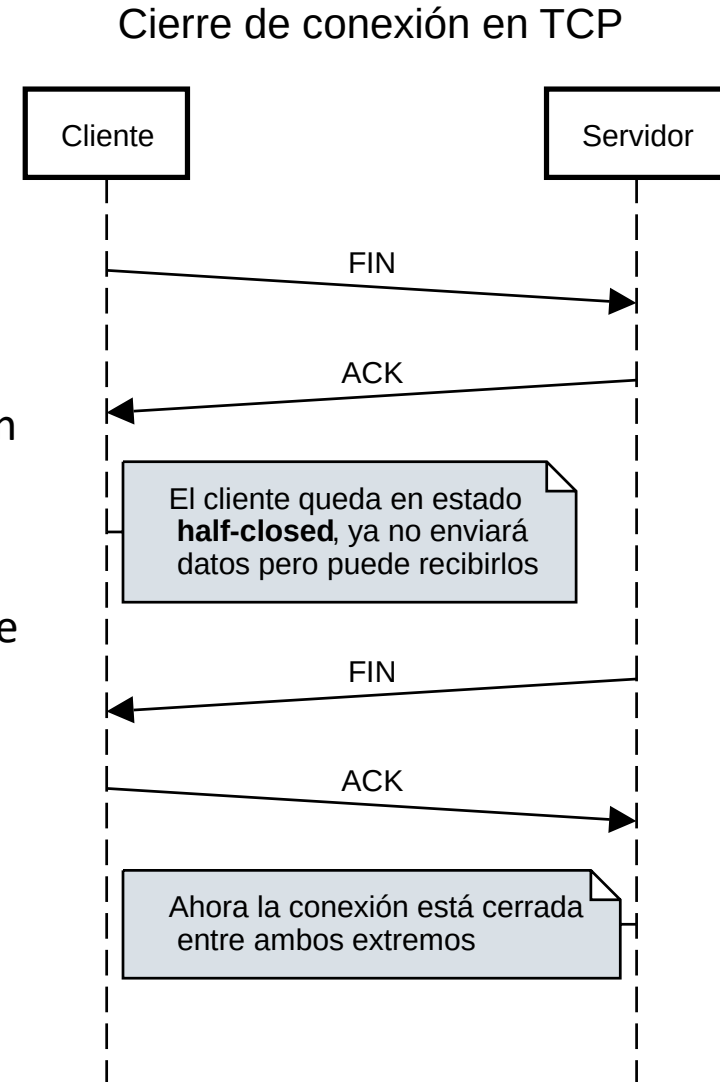
Descripción

Cualquiera de los dos extremos puede iniciar el proceso de cierre de conexión cuando ya no tiene más datos que enviar a la otra parte

Secuencia de cierre de conexión:

1. Una de las partes envía un segmento con el bit **F (IN)** activado y solicita la terminación
2. La otra parte puede enviar un segmento con el bit **A (CK)** para reconocer la petición, pero si no tiene más datos que enviar también podría enviar un segmento **F (IN)**
3. La parte que envía la petición de cierre y recibe el **A (CK)** queda en un estado en el que no enviará datos, pero sí podría todavía recibirlos de la otra parte
4. Una vez que la otra parte envía su segmento **F (IN)**, el extremo que solicitó el cierre envía su segmento **A (CK)** y la conexión queda finalmente cerrada

Finalizada la conexión, ya no pueden intercambiarse datos entre los *host* hasta que se inicie una nueva conexión.



Comparativa entre UDP y TCP



El protocolo TCP - Comparativa con UDP

Definición

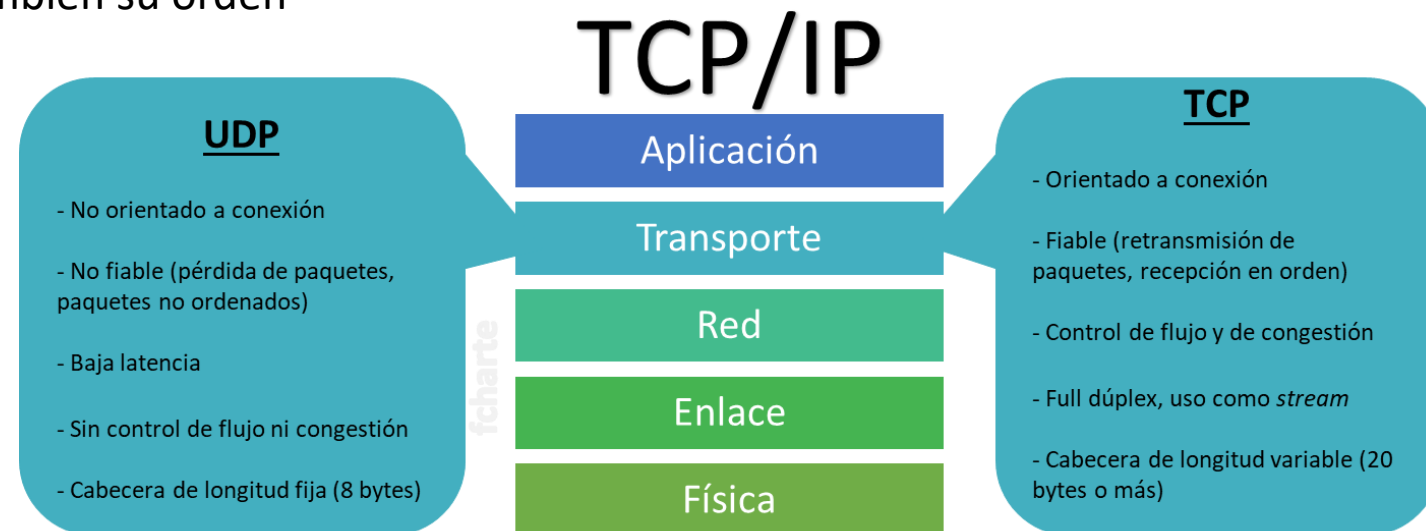
TCP es un protocolo de transporte más complejo que UDP y usa un segmento con más campos, ofreciendo a cambio características avanzadas

Detalles

- Precisa un **establecimiento de conexión** antes de poder enviar o recibir datos
- Mantiene unas variables de **estado** de conexión tanto en emisor como en receptor
- El segmento TCP cuenta con una **cabecera de mayor tamaño**, por lo que es menos eficiente
- El control de congestión evita que el **emisor desborde** al receptor
- Se **garantiza la entrega** de los segmentos y también su orden

Usos

- TCP se usa en todos aquellos escenarios donde **no hay tolerancia** a la pérdida de datos
 - Protocolos de correo: SMTP, POP3 e IMAP
 - Intercambio de archivos y web: FTP y HTTP
- **No es el protocolo adecuado** cuando se requiere baja latencia y alta eficiencia



Multiplexación en UDP y TCP



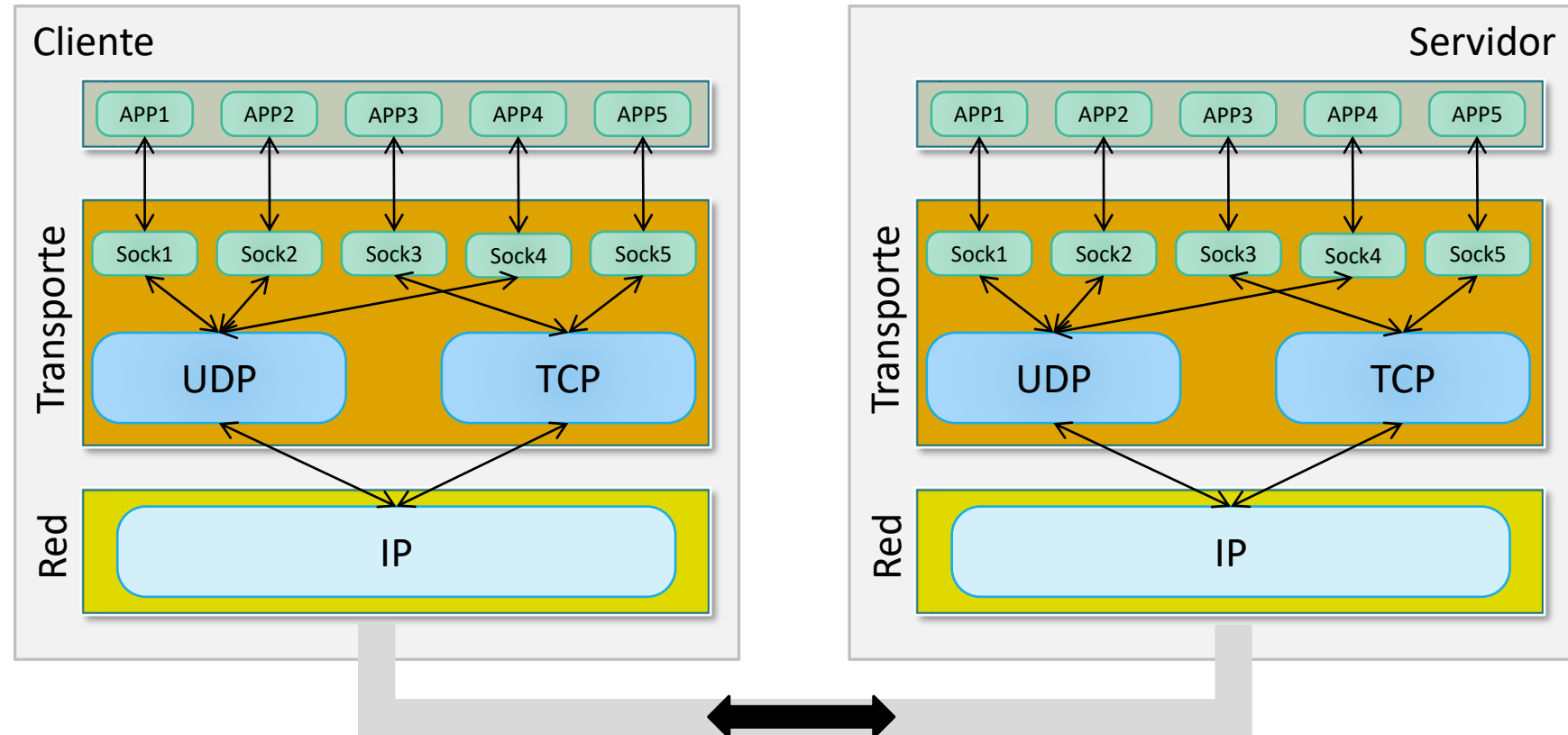
Mecanismo de multiplexación

Problemática

En un host múltiples procesos de la capa de aplicación demandan servicios de comunicación, pero existe una única conexión de red física

Solución:

- Sistema de **multiplexación** basado en direcciones de la capa de transporte (**puertos**) que permita la entrega correcta de datos al proceso de la capa de aplicación que corresponda
- Un **socket** es el punto de acceso a los servicios de la capa de transporte, usados por los procesos de la capa de aplicación



Demultiplexación en la capa de transporte

Objetivo

Redirigir los datagramas IP que se reciben en la capa de red, a través de una única conexión, al proceso adecuado de la capa de aplicación

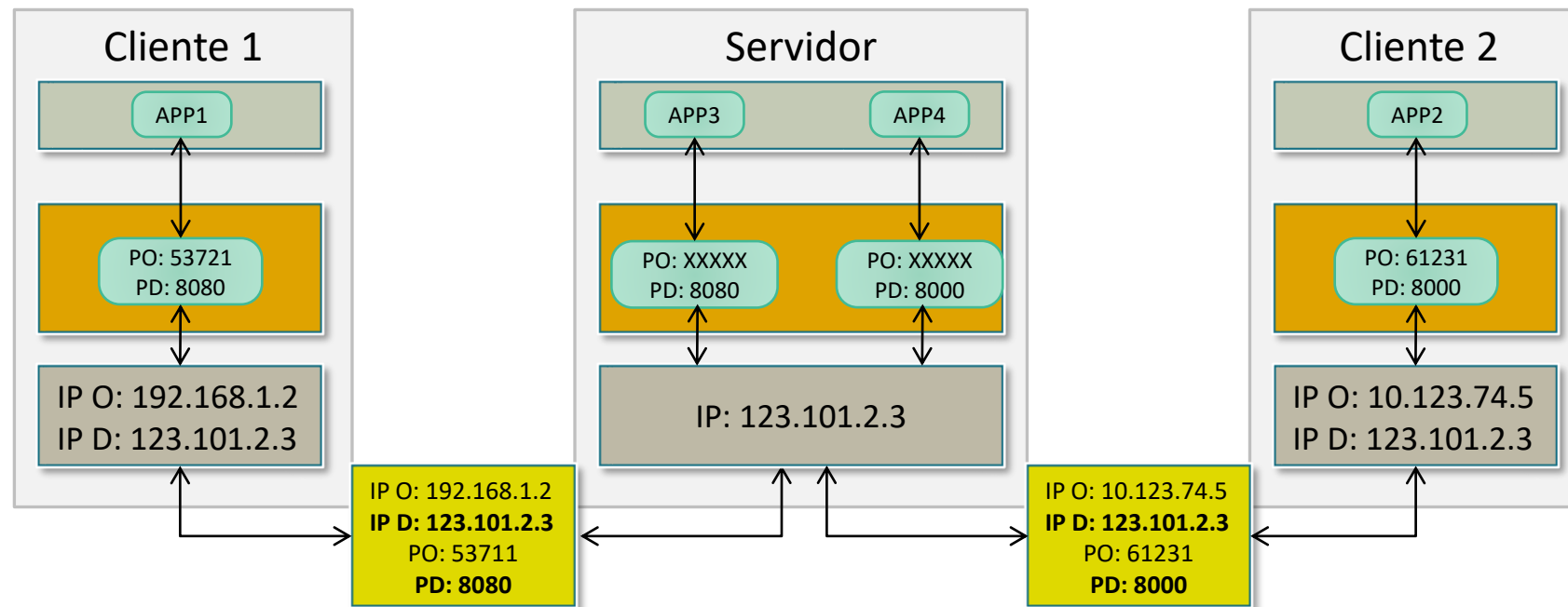
Detalles

- Cada datagrama tiene una **dirección IP** origen y una dirección IP destino
- Cada datagrama **transporta un segmento** de la capa de transporte
- Cada segmento tiene un **número de puerto** origen y un número de destino

El *host* utiliza las direcciones IP y números de puerto para **dirigir los segmentos** hacia el *socket* que les corresponda

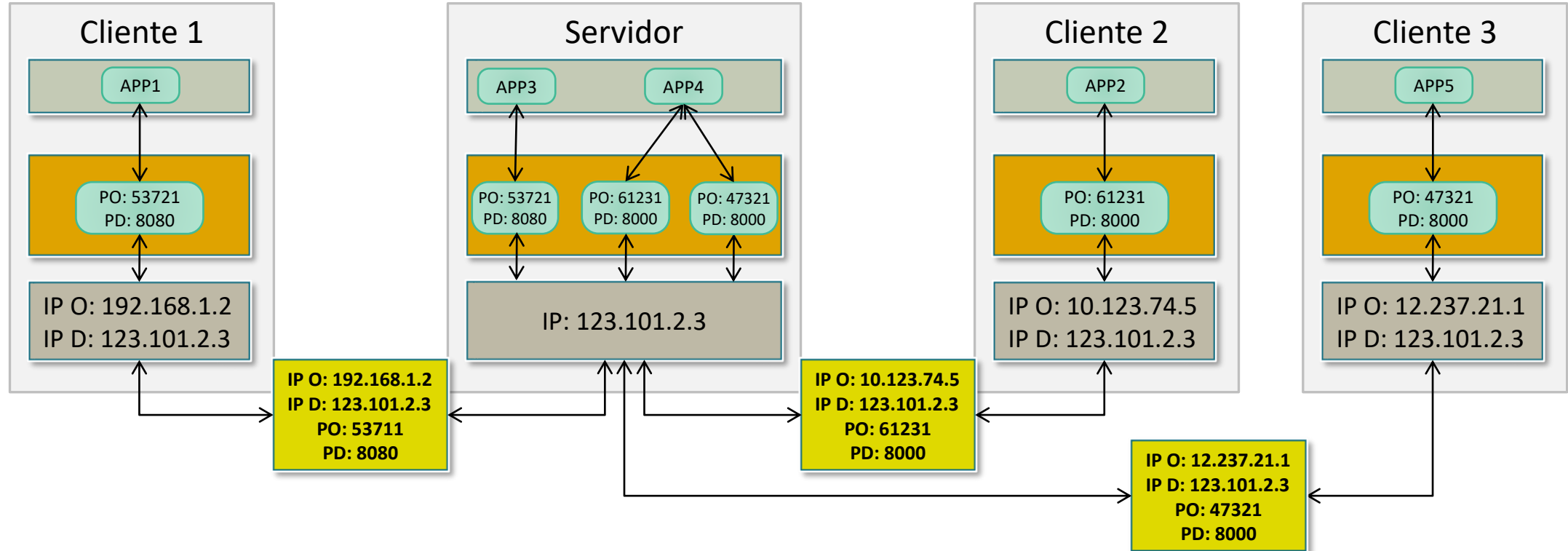
Demultiplexación con el protocolo UDP

- *Socket* UDP identificado por el par: (IP destino, Puerto destino)
- **No existe conexión**, solo envío/recepción de segmentos UDP independientes entre sí
- El puerto de origen proporciona los datos de la **dirección de retorno** si hubiese que responder
- Cuando la capa de transporte del *host* destino recibe el segmento UDP:
 - Revisa el número de puerto destino en el segmento
 - Dirige el segmento UDP al *socket* con ese número de puerto



Demultiplexación con el protocolo TCP

- *Socket* TCP identificado por la tupla: (IP origen, Puerto origen, IP destino, Puerto destino)
- **Se establece una conexión** y cada una de ellas se identifica por la anterior tupla, por lo que:
 - El servidor puede mantener múltiples *sockets* activos asociados a la misma dirección y el mismo puerto de destino
 - Esto permite que una misma aplicación en el servidor atienda a múltiples clientes sin problemas
- El receptor usa la tupla para identificar de manera única la conexión que pertenece cada segmento recibido



Actividades y recursos adicionales



Atendiendo al formato del segmento UDP, intenta responder a las siguientes cuestiones:

- ¿Cuántos procesos podrían **utilizar UDP simultáneamente**? Ten en cuenta que cada proceso precisa un puerto
- ¿Cuál es teóricamente la **cantidad máxima de información** que puede enviarse con UDP?
- Asumiendo que usamos UDP para enviar un mensaje que ocupa 248 bytes, calcula cuál sería la **eficiencia del protocolo** en ese caso concreto

Atendiendo al funcionamiento del protocolo UDP, razona la siguiente cuestión:

- ¿Podría establecerse una **conexión fiable** usando UDP?



Atendiendo al formato del segmento TCP, sabiendo que la MTU de Ethernet es de **1500 bytes** y que la cabecera IP tiene un tamaño de **20 bytes**, intenta responder a las siguientes cuestiones:

- ¿Cuál sería el tamaño máximo del *payload* de un segmento TCP que tuviera **8 bytes** en el campo opciones?
- Suponiendo que por dicha conexión TCP (manteniendo los 8 bytes en el campo opciones) vayamos a enviar un archivo que ocupa **2920 bytes**, ¿cuántos paquetes tendrán que enviarse?
- Asumiendo que el campo **SEQ** de establecimiento de conexión tenga el valor inicial **100**, ¿cuáles serán los números de secuencia de los paquetes en que se enviarán las porciones de dicho archivo?

Instalación de herramientas en Ubuntu

- Si no lo hiciste la semana anterior, procede a instalar la utilidad hping3 en tu máquina:
 - `sudo apt update`
 - `sudo apt install hping3`

Comprobación de MSS

- Ejecuta la siguiente orden cambiando el valor del parámetro `-d` para ajustar la cantidad de datos a enviar. Si el paquete de datos supera el tamaño del MSS los paquetes se perderán
 - `sudo hping3 -V -c 1 -d 1400 --dontfrag fcharte.com`

Números secuencia

- Ejecuta la siguiente orden. El parámetro `-M` establece el número de secuencia inicial. Observa cómo se incrementa en la respuesta, en la que aparece como `ack`, y el número de secuencia enviado por la otra parte
 - `sudo hping3 -V -c 1 -p 80 -S -M 1000 fcharte.com`

```
usuario@par:~$ sudo hping3 -V -c 1 -d 1400 --dontfrag fcharte.com
using enp0s3, addr: 10.0.2.15, MTU: 1500
HPING fcharte.com (enp0s3 185.199.108.153): NO FLAGS are set, 40 headers + 1400 data bytes
len=46 ip=185.199.108.153 ttl=255 id=714 tos=0 iplen=40
sport=0 flags=RA seq=0 win=0 rtt=7.6 ms
seq=0 ack=494135893 sum=5c6b urp=0

--- fcharte.com hping statistic ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 7.6/7.6/7.6 ms
usuario@par:~$
usuario@par:~$ sudo hping3 -V -c 1 -d 1480 --dontfrag fcharte.com
using enp0s3, addr: 10.0.2.15, MTU: 1500
HPING fcharte.com (enp0s3 185.199.108.153): NO FLAGS are set, 40 headers + 1480 data bytes
auto-activate fragmentation, fragments size: 1480

--- fcharte.com hping statistic ---
1 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
usuario@par:~$
usuario@par:~$ sudo hping3 -V -c 1 -p 80 -S -M 1000 fcharte.com
using enp0s3, addr: 10.0.2.15, MTU: 1500
HPING fcharte.com (enp0s3 185.199.108.153): S set, 40 headers + 0 data bytes
len=46 ip=185.199.108.153 ttl=64 id=720 tos=0 iplen=44
sport=80 flags=SA seq=0 win=65535 rtt=30.0 ms
seq=25600001 ack=1001 sum=e1b6 urp=0

--- fcharte.com hping statistic ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 30.0/30.0/30.0 ms
usuario@par:~$
```


Instalación de herramientas en Ubuntu

- Si no lo hiciste antes, ejecuta los siguientes comandos para instalar la herramienta hping3 en tu máquina:
 - `sudo apt update`
 - `sudo apt install hping3`

Barrido de puertos

- Para efectuar un barrido de puertos en el equipo propio ejecutaremos la siguiente orden:
 - `sudo hping3 --scan 0-4095 -S localhost`
- Con el parámetro `--scan` se indica el rango de puertos a examinar, en este ejemplo los primeros 4096
- La opción `-S` hace que en los segmentos enviados se active el bit **SYN**, simulando que queremos abrir una conexión
- Agrega la opción `-V` para ver la respuesta para cada uno de los puertos. En su mayor parte verás que devuelven un segmento con el bit **R(ESET)** activado, indicando que no hay proceso a la escucha en ese puerto
- ¿Cuántos puertos hay abiertos en tu sistema? ¿Cuál es su finalidad?

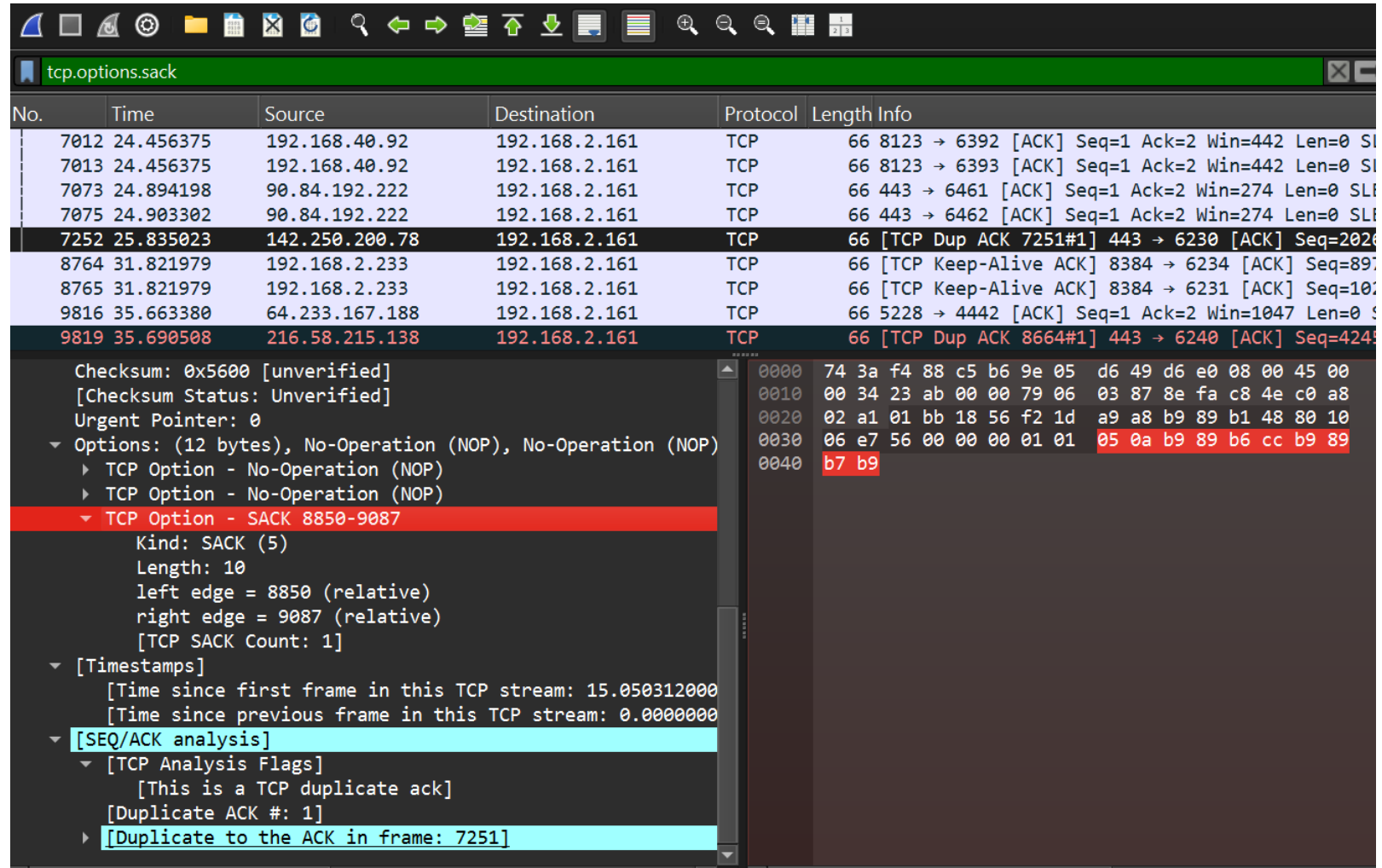
```
[usuario@servidor ~]$  
[usuario@servidor ~]$ sudo hping3 --scan 0-4095 -S localhost  
Scanning localhost (127.0.0.1), port 0-4095  
4096 ports to scan, use -V to see all the replies  
+-----+  
|port| serv name | flags | ttl | id | win | len |  
+-----+  
22  ssh      : .S..A... 64    0 43690 44  
25  smtp     : .S..A... 64    0 43690 44  
111 sunrpc   : .S..A... 64    0 43690 44  
2521 adaptecMgr : .S..... 64 42013 512 40  
All replies received. Done.  
Not responding ports:  
[usuario@servidor ~]$ _
```

Wireshark

Utiliza Wireshark para capturar el tráfico de tu interfaz de red y, a continuación, establece una conexión TCP, por ejemplo accediendo a una página web, un servidor FTP o de correo. Realiza distintas operaciones que permitan capturar una cantidad significativa de paquetes.

Filtra los paquetes para buscar aquellos en los que está presente la opción `sack`, tal y como se aprecia en la imagen de la derecha. Esto te permitirá comprobar si se han recibido segmentos ACK duplicados y, en dicho caso, si se ha recurrido a SACK para informar de las partes que ya han llegado al receptor.

En el ejemplo de la derecha el equipo con IP 142.250.200.78 informa al que tiene IP 192.168.2.161 que ya ha recibido los segmentos con números de secuencia de 8850 a 9087, habiéndose incluido esta opción porque se ha recibido un segmento fuera de orden.



The image shows a Wireshark packet capture window with the filter `tcp.options.sack` applied. The packet list on the left shows several TCP packets, with packet 7252 (No. 7252, Time 25.835023, Source 142.250.200.78, Destination 192.168.2.161) selected. The packet details pane on the right shows the selected packet's structure, including the TCP header and options. The SACK option is expanded, showing the left edge (8850) and right edge (9087) relative to the sequence number. The packet bytes pane on the right shows the raw data of the packet, with the SACK option bytes highlighted in red.

No.	Time	Source	Destination	Protocol	Length	Info
7012	24.456375	192.168.40.92	192.168.2.161	TCP	66	8123 → 6392 [ACK] Seq=1 Ack=2 Win=442 Len=0 SI
7013	24.456375	192.168.40.92	192.168.2.161	TCP	66	8123 → 6393 [ACK] Seq=1 Ack=2 Win=442 Len=0 SI
7073	24.894198	90.84.192.222	192.168.2.161	TCP	66	443 → 6461 [ACK] Seq=1 Ack=2 Win=274 Len=0 SLI
7075	24.903302	90.84.192.222	192.168.2.161	TCP	66	443 → 6462 [ACK] Seq=1 Ack=2 Win=274 Len=0 SLI
7252	25.835023	142.250.200.78	192.168.2.161	TCP	66	[TCP Dup ACK 7251#1] 443 → 6230 [ACK] Seq=2026
8764	31.821979	192.168.2.233	192.168.2.161	TCP	66	[TCP Keep-Alive ACK] 8384 → 6234 [ACK] Seq=897
8765	31.821979	192.168.2.233	192.168.2.161	TCP	66	[TCP Keep-Alive ACK] 8384 → 6231 [ACK] Seq=107
9816	35.663380	64.233.167.188	192.168.2.161	TCP	66	5228 → 4442 [ACK] Seq=1 Ack=2 Win=1047 Len=0 S
9819	35.690508	216.58.215.138	192.168.2.161	TCP	66	[TCP Dup ACK 8664#1] 443 → 6240 [ACK] Seq=4243

Checksum: 0x5600 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
Options: (12 bytes), No-Operation (NOP), No-Operation (NOP)
 ▶ TCP Option - No-Operation (NOP)
 ▶ TCP Option - No-Operation (NOP)
 ▼ TCP Option - SACK 8850-9087
 Kind: SACK (5)
 Length: 10
 left edge = 8850 (relative)
 right edge = 9087 (relative)
 [TCP SACK Count: 1]
 ▼ [Timestamps]
 [Time since first frame in this TCP stream: 15.050312000]
 [Time since previous frame in this TCP stream: 0.0000000]
 ▼ [SEQ/ACK analysis]
 ▼ [TCP Analysis Flags]
 [This is a TCP duplicate ack]
 [Duplicate ACK #: 1]
 ▶ [Duplicate to the ACK in frame: 7251]

Material adicional

Descripción

Para ampliar tus conocimientos sobre los contenidos de esta semana te recomendamos que consultes los recursos indicados a continuación.

Recursos

- **Capítulo 3** La capa de transporte, del libro Redes de computadoras 7ED disponible en [formato digital](#) en la BUJA (recuerda identificarte para poder acceder a leerlo desde tu navegador), concretamente las **secciones 3.1 a la 3.5**
- **TCP/IP Transport Layer Protocolos** en el recurso electrónico [The TCP/IP Guide](#), donde encontrarás todos los detalles sobre el funcionamiento de UDP y TCP
- **hping Home** (<http://www.hping.org>) es la [página oficial](#) de la herramienta hping3, en la que encontrarás enlaces a descargas para otros sistemas, documentación, tutoriales, etc.

Cuestiones clave de este tema



Cuestiones clave

Qué deberías saber

*Al inicio de este tema se planteaban unos objetivos específicos que deberían permitirte **responder a las siguientes cuestiones clave***

Cuestiones

- ¿Qué servicios ofrece la capa de transporte a la capa de aplicación?
- ¿Cómo funciona el protocolo UDP?
- ¿Cuál es el formato del segmento UDP?
- ¿Cómo funciona el protocolo TCP?
- ¿Cuál es el formato del segmento TCP?
- ¿Cuáles son los pasos del establecimiento de conexión y determinación del MSS?
- ¿Cómo funciona la multiplexación/demultiplexación en UDP y TCP?